

Trabalho Prático - Steinway

Unidade Curricular: Bases de Dados e Armazéns de Dados

Turma: 2DA

Grupo de Trabalho:

Miguel Silva – 1241131

Pedro Silva – 1242036

Lucas Santos – 1241008

Ano Letivo 2025/2026

Conteúdo

1	Introdução	3
2	Modelo Dimensional	3
3	Estrutura da Staging Area	4
4	Processos de ETL	5
5	Scripts SQL	5
5.1	<i>Staging Area</i>	6
5.1.1	Tabela <i>Customers</i>	6
5.1.2	Tabela <i>CustomersDQP</i>	6
5.1.3	Tabela <i>CustomerTypes</i>	7
5.1.4	Tabela <i>Departaments</i>	7
5.1.5	Tabela <i>Employees</i>	7
5.1.6	Tabela <i>EmployeesDQP</i>	8
5.1.7	Tabela <i>Families</i>	8
5.1.8	Tabela <i>Products</i>	8
5.1.9	Tabela <i>ProductsDQP</i>	9
5.1.10	Tabela <i>Sales</i>	9
5.1.11	Tabela <i>SalesDetails</i>	9
5.1.12	Tabela <i>FactSalesDQP</i>	10
5.2	<i>Data Mart</i>	10
5.2.1	Dimensão <i>DimCostumer</i>	10
5.2.2	Dimensão <i>DimDate</i>	11
5.2.3	Dimensão <i>DimEmployee</i>	12
5.2.4	Dimensão <i>DimProduct</i>	12
5.2.5	Tabela de Factos (<i>FactSales</i>)	13
6	Conclusões	14

1 Introdução

Este relatório descreve o processo de criação, e as suas respetivas tomadas de decisão, de um armazém de dados, relativo à empresa *Steinway*. Será abordado o modelo dimensional desenvolvido; a comparação entre a estrutura de dados original e a do Data Mart, juntamente com os processos de ETL necessários para garantir a melhor qualidade do resultado final. Por fim, escrutinar os scripts SQL das tabelas, para uma maior compreensão da estrutura dos dados.

Devemos alertar que, como este projeto foi desenvolvido utilizando um repositório do GitHub, por motivos colaborativos, a caminho do projeto será `C:\Temp\TrabPrat\TrabPrat`. Simplificando, o projeto deve ser executado com a estrutura de diretórios original que se encontra no ficheiro ZIP.

2 Modelo Dimensional

O Modelo Dimensional, presente na figura 1, foi desenhado com base na estrutura das tabelas da fonte de origem, *Steinway*, e das suas relações de chave primárias e estrangeiras.

Para garantir um maior nível de granularidade, foram selecionadas três tabelas da fonte para serem dimensões: *Customers*, *Products* e *Employees*; sendo transformadas em *DimCustomer*, *DimProduct* e *DimEmployee*, respetivamente.

A dimensão *DimCustomer* é uma junção das tabelas *Customers* e *CustomerTypes*, a *DimProduct* junta *Product* e *Families*, e a dimensão *DimEmployee* junta *Employees* e *Departaments*. Com esta simplificação, no número de tabelas, conseguimos reduzir a complexidade do armazém de dados e, por conseguinte, aumentar o nível de granularidade, como previsto.

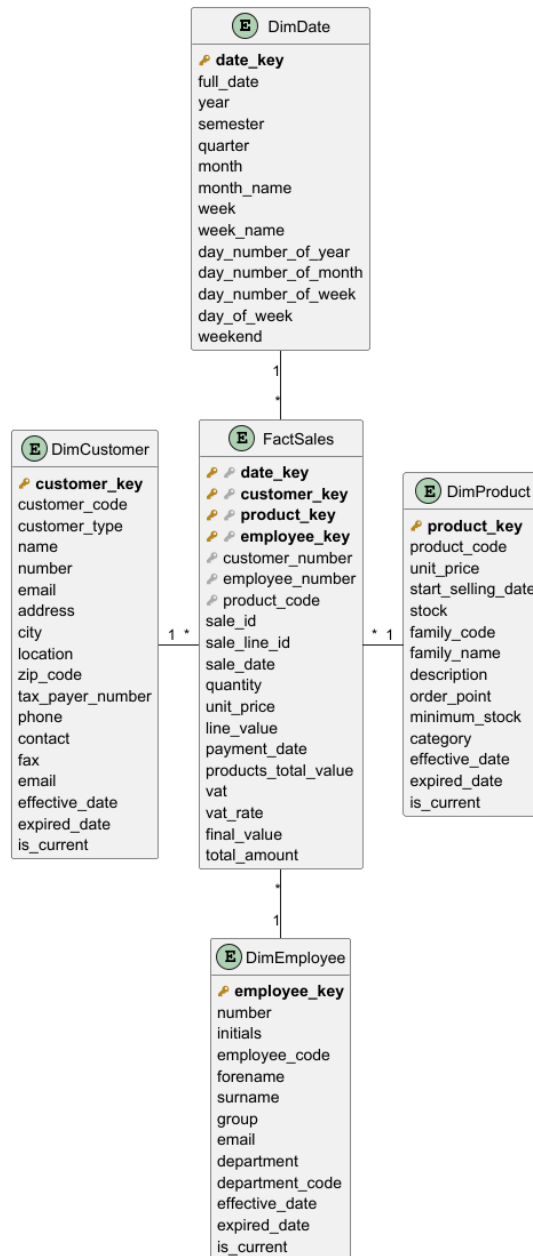


Figura 1: Modelo Dimensional

3 Estrutura da Staging Area

Na *Staging Area* a estrutura dos dados inicialmente foi realizada através do carregamento dos dados provenientes da *Steinway*. Para uma melhor organização do futuro *Data Mart* foram feitas as seguintes alterações:

- Remoção das chaves primárias e estrangeiras, assegurando o desacoplamento das tabelas.
- Criação de tabelas DQP (*Data Quality Problems*) para prevenir possíveis erros, garantindo a qualidade dos dados do Data Mart.

4 Processos de ETL

Para garantir uma maior precisão nos processos de ETL, seguimos as indicações dadas nos ficheiros provenientes da cadeia *Base de Dados e Armazéns de Dados* redirecionadas ao contexto do nosso projeto.

Processos estes que foram aplicados a todas as dimensões, desde verificação da existência de números negativos a propriedades vazias, posteriormente traduzidas para "Unknown". A seguinte lista representa os processos de ETL embutidos nas nossas dimensões e na tabela de factos:

- *DimCustomer*:
 - Tamanho do nome tem de ser maior que zero
 - Email deve conter um "@"e um "."após
- *DimEmployee*:
 - Tamanho das iniciais deve ser maior que zero
 - Tamanho do nome próprio deve ser maior que zero
 - Tamanho do apelido deve ser maior que zero
 - Email deve conter um "@"e um "."após
- *DimProduct*:
 - Quantidade de stock deve ser maior que zero
 - Quantido de stock mínimo deve ser maior que zero
 - Preço por unidade deve ser maior que zero
- *FactSales*:
 - A data de pagamento deve ser após, ou no mesmo dia, que a data de compra
 - Quantidade deve ser maior que zero
 - Preço por unidade deve ser maior que zero
 - O valor do imposto deve ser maior, ou igual, que zero
 - A taxa de imposto deve ser maior, ou igual, que zero

5 Scripts SQL

Por último, demonstramos abaixo os scripts para a criação das tabelas do nosso projeto.

De uma forma geral, entre as tabelas da *Steinway* e da *Staging Area* as mudanças foram, principalmente: a remoção de relações (chaves primárias e estrangeiras), remoção da propriedade *IDENTITY* nos identificadores e a criação de tabelas DQP (*Data Quality Problems*) para afastar a possibilidade de tuplos defeituosos serem guardados no Data Mart.

5.1 *Staging Area*

5.1.1 Tabela *Customers*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'Customers'
2 )
3     create table Customers
4     (
5         Name          varchar(55) not null,
6         Number        numeric(10) not null,
7         TaxpayerNumber varchar(20) not null,
8         Fax           varchar(60) not null,
9         Phone         varchar(60) not null,
10        Contact       varchar(30) not null,
11        Address       varchar(55) not null,
12        ZipCode       varchar(10) not null,
13        City          varchar(33),
14        Location      varchar(43) not null,
15        CustomerType  int         not null,
16        Email         varchar(45) not null
17    )
18 ELSE
19     DELETE FROM Customers;
```

5.1.2 Tabela *CustomersDQP*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
2 CustomersDQP')
3     create table CustomersDQP
4     (
5         Name          varchar(55) not null,
6         Number        numeric(10) not null,
7         TaxpayerNumber varchar(20) not null,
8         Fax           varchar(60) not null,
9         Phone         varchar(60) not null,
10        Contact       varchar(30) not null,
11        Address       varchar(55) not null,
12        ZipCode       varchar(10) not null,
13        City          varchar(33),
14        Location      varchar(43) not null,
15        CustomerType  int         not null,
16        Email         varchar(45) not null,
17        DQP           nvarchar(100)
18    )
19 ELSE
20     DELETE FROM CustomersDQP;
```

5.1.3 Tabela *CustomerTypes*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
  CustomerTypes')
2   create table CustomerTypes
3   (
4       Code int,
5       Type char(20) not null
6   )
7 ELSE
8   DELETE FROM CustomerTypes;
```

5.1.4 Tabela *Departaments*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
  Departaments')
2   create table Departaments
3   (
4       Code int,
5       Department varchar(50) not null
6   )
7 ELSE
8   DELETE FROM Departaments;
```

5.1.5 Tabela *Employees*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'Employees'
  )
2   create table Employees
3   (
4       Number numeric(6) not null,
5       Initials varchar(3) not null,
6       Code varchar(20) not null,
7       Forename varchar(50) not null,
8       Surname varchar(50) not null,
9       [Group] varchar(20) not null,
10      Department int not null,
11      Email varchar(100) not null
12  )
13 ELSE
14   DELETE FROM Employees;
```

5.1.6 Tabela *EmployeesDQP*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
  EmployeesDQP')
2   create table EmployeesDQP
3   (
4       Number      numeric(6)      not null,
5       Initials    varchar(3)      not null,
6       Code        varchar(20)     not null,
7       Forename    varchar(50)     not null,
8       Surname     varchar(50)     not null,
9       [Group]     varchar(20)     not null,
10      Department  int              not null,
11      Email       varchar(100)     not null,
12      DQP         nvarchar(100)
13  )
14 ELSE
15   DELETE FROM EmployeesDQP;
```

5.1.7 Tabela *Families*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'Families')
2   create table Families
3   (
4       Code int,
5       Name varchar(60)
6   )
7 ELSE
8   DELETE FROM Families;
```

5.1.8 Tabela *Products*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'Products')
2   create table Products
3   (
4       Code          char(18)      not null,
5       Description   char(60)      not null,
6       FamilyCode    int           not null,
7       Stock         numeric(13, 3) not null,
8       UnitPrice     numeric(19, 6) not null,
9       OrderPoint    numeric(10, 3) not null,
10      MinimunStock   numeric(13, 3) not null,
11      StartSellingDate date        not null,
12      Category       varchar(25)   not null
13  )
14 ELSE
15   DELETE FROM Products;
```

5.1.9 Tabela *ProductsDQP*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
  ProductsDQP')
2   create table ProductsDQP
3   (
4       Code          char(18)          not null,
5       Description    char(60)          not null,
6       FamilyCode     int              not null,
7       Stock          numeric(13, 3)    not null,
8       UnitPrice       numeric(19, 6)    not null,
9       OrderPoint     numeric(10, 3)    not null,
10      MinimunStock    numeric(13, 3)    not null,
11      StartSellingDate date            not null,
12      Category        varchar(25)      not null,
13      DQP nvarchar(100)
14  )
15 ELSE
16   DELETE FROM ProductsDQP;
```

5.1.10 Tabela *Sales*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'Sales')
2   create table Sales
3   (
4       SaleID          int,
5       SaleDate         date            not null,
6       CustomerNumber   numeric(10)     not null,
7       EmployeeNumber   numeric(6)      not null,
8       PaymentDate      date            not null,
9       ProductsTotalValue numeric(19, 6) not null,
10      VAT               numeric(19, 6)  not null,
11      FinalValue        numeric(19, 6)  not null
12  )
13 ELSE
14   DELETE FROM Sales;
```

5.1.11 Tabela *SalesDetails*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
  SalesDetails')
2   create table SalesDetails
3   (
4       SaleID          int              not null,
5       SaleLineID      int              not null,
6       ProductCode     char(18)         not null,
7       Quantity        numeric(14, 4)   not null,
8       VATRate         numeric(4, 2)    not null,
9       UnitPrice       numeric(19, 6)    not null,
```

```

10         LineValue    numeric(19, 6) not null,
11     )
12 ELSE
13     TRUNCATE TABLE SalesDetails

```

5.1.12 Tabela *FactSalesDQP*

```

1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
   FactSalesDQP')
2     create table FactSalesDQP
3     (
4         SaleID          int,
5         SaleDate         date          not null,
6         CustomerNumber   numeric(10)   not null,
7         EmployeeNumber   numeric(6)    not null,
8         PaymentDate      date          not null,
9         ProductsTotalValue numeric(19, 6) not null,
10        VAT              numeric(19, 6) not null,
11        FinalValue       numeric(19, 6) not null,
12        SaleLineID       int            not null,
13        Quantity         numeric(14, 4) not null,
14        VATRate          numeric(4, 2)  not null,
15        UnitPrice        numeric(19, 6) not null,
16        LineValue        numeric(19, 6) not null,
17        DQP              nvarchar(100)
18    )
19 ELSE
20     DELETE FROM FactSalesDQP;

```

5.2 Data Mart

5.2.1 Dimensão *DimCostumer*

```

1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
   DimCustomer')
2 BEGIN
3     CREATE TABLE DimCustomer
4     (
5         customer_key int IDENTITY(1,1) NOT NULL,
6         name varchar(55) NOT NULL,
7         number numeric(10) UNIQUE NOT NULL,
8         email varchar(45) NOT NULL,
9         address varchar(55) NOT NULL,
10        city varchar(33),
11        location varchar(43) NOT NULL,
12        zip_code varchar(10) NOT NULL,
13        tax_payer_number varchar(20) NOT NULL,
14        phone varchar(60) NOT NULL,
15        contact varchar(30) NOT NULL,
16        fax varchar(60) NOT NULL,

```

```

17     customer_code int,
18     customer_type char(20) NOT NULL,
19     effective_date date NOT NULL,
20     expired_date date,
21     is_current bit NOT NULL,
22     CONSTRAINT [PK_DimCustomer] PRIMARY KEY CLUSTERED
23 (
24     customer_key ASC
25 )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
26 IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS =
27 ON) ON [PRIMARY]
28 CREATE NONCLUSTERED INDEX [NonClusteredIndex-CustomerNumber] ON
29 [dbo].[DimCustomer]
30 (
31     [number] ASC
32 )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
33 SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF,
34 ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON,
35 OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
36
37 END

```

5.2.2 Dimensão *DimDate*

```

1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'DimDate')
2 CREATE TABLE [dbo].[DimDate](
3     [DateKey] [int] IDENTITY(1,1) NOT NULL,
4     [FullDate] [datetime] NOT NULL,
5     [Year] [int] NOT NULL,
6     [Semester] [tinyint] NOT NULL,
7     [Quarter] [tinyint] NOT NULL,
8     [Month] [tinyint] NOT NULL,
9     [MonthName] [nvarchar](10) NOT NULL,
10    [Week] [tinyint] NOT NULL,
11    [DayNumberOfYear] [int] NOT NULL,
12    [DayNumberOfMonth] [tinyint] NOT NULL,
13    [DayNumberOfWeek] [tinyint] NOT NULL,
14    [DayOfWeek] [nvarchar](10) NOT NULL,
15    [Weekend] [nvarchar](3) NOT NULL,
16    CONSTRAINT [PK_DimDate] PRIMARY KEY CLUSTERED
17 (
18     [DateKey] ASC
19 )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
20 IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS =
21 ON) ON [PRIMARY]
22 ) ON [PRIMARY]

```

5.2.3 Dimensão *DimEmployee*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = '
   DimEmployee')
2 BEGIN
3     CREATE TABLE DimEmployee
4     (
5         employee_key int IDENTITY(1,1) NOT NULL,
6         number numeric(6) UNIQUE NOT NULL,
7         initials varchar(3) NOT NULL,
8         employee_code varchar(20) NOT NULL,
9         forename varchar(50) NOT NULL,
10        surname varchar(50) NOT NULL,
11        [group] varchar(20) NOT NULL,
12        email varchar(100) NOT NULL,
13        department_code int,
14        department varchar(50) NOT NULL,
15        effective_date date NOT NULL,
16        expired_date date,
17        is_current bit NOT NULL,
18        CONSTRAINT [PK_DimEmployee] PRIMARY KEY CLUSTERED
19        (
20            employee_key ASC
21        )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
   IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS =
   ON) ON [PRIMARY]
22    ) ON [PRIMARY]
23    create nonclustered index [NonClusteredIndex-EmployeeNumber] on
   [dbo].[DimEmployee]
24    (
25        [number] ASC
26    )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
   SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF,
   ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
27 END
```

5.2.4 Dimensão *DimProduct*

```
1 IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'DimProduct
   ')
2 BEGIN
3     CREATE TABLE DimProduct
4     (
5         product_key int IDENTITY(1,1) NOT NULL,
6         product_code char(18) UNIQUE NOT NULL,
7         description varchar(60) NOT NULL,
8         stock numeric(13,3) NOT NULL,
9         unit_price numeric(19,6) NOT NULL,
10        order_point numeric(10,3) NOT NULL,
11        minimum_stock numeric(13,3) NOT NULL,
12        start_selling_date date NOT NULL,
```

```

13      category varchar(25) NOT NULL,
14      family_code int,
15      family_name varchar(60) NOT NULL,
16      effective_date date NOT NULL,
17      expired_date date,
18      is_current bit NOT NULL,
19      CONSTRAINT [PK_DimProduct] PRIMARY KEY CLUSTERED
20  (
21      product_key ASC
22  )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
23  IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS =
24  ON) ON [PRIMARY]
25  ) ON [PRIMARY]
26  create nonclustered index [NonClusteredIndex-ProductCode] on [
27  dbo].[DimProduct]
28  (
29      [product_code] asc
30  )WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
31  SORT_IN_TEMPDB = OFF, DROP_EXISTING = OFF, ONLINE = OFF,
32  ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) on [PRIMARY]
33  END

```

5.2.5 Tabela de Factos (*FactSales*)

```

1  IF NOT EXISTS (SELECT name FROM sys.tables WHERE name = 'FactSales'
2  )
3  CREATE TABLE FactSales
4  (
5      [DateKey] [int] NOT NULL,
6      customer_key int NOT NULL,
7      product_key [int] NOT NULL,
8      employee_key int NOT NULL,
9      sale_id int NOT NULL,
10     sale_line_id int NOT NULL,
11     sale_date date NOT NULL,
12     customer_number numeric(10) NOT NULL
13         constraint FK_DM_Sales_Customers
14         references DimCustomer(number),
15     employee_number numeric(6) NOT NULL
16         constraint FK_DM_Sales_Employees
17         references DimEmployee(number),
18     product_code char(18) NOT NULL
19         constraint FK_DM_Sales_Product
20         references DimProduct(product_code),
21     quantity numeric(14,4) NOT NULL,
22     unit_price numeric(19,6) NOT NULL,
23     payment_date date NOT NULL,
24     products_total_value numeric(19,6) NOT NULL,
25     vat numeric(19,6) NOT NULL,
26     vat_rate numeric(4,2) NOT NULL,
27     line_value numeric(19,6) NOT NULL,

```

```

27      final_value numeric(19,6) NOT NULL,
28      total_amount numeric(14,4)
29  CONSTRAINT [PK_FactSales] PRIMARY KEY CLUSTERED
30  (
31      [DateKey] ASC,
32      customer_key ASC,
33      product_key ASC,
34      employee_key ASC
35  ) WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF,
36  IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS =
  ON) ON [PRIMARY]
  ) ON [PRIMARY]

```

6 Conclusões

Este armazém de dados cumpre, eficientemente, a premissa de desenvolver um *Data Mart* capaz de realizar consultas aos dados das vendas no nível de granularidade mais elementar. Conseguindo, portanto, fazer análises a maior flexibilidade possível.

Possíveis melhorias, neste trabalho, seriam a implementação de mais restrições à qualidade dos dados introduzidos no Data Mart e mensagens de erro mais descritivas, aquando da passagem de dados para as tabelas de DQP. No entanto, decidimos, em grupo, não ser tão rígidos e permitir alguma flexibilidade.